An Interactive Orrery Simulation for Learning Celestial Mechanics in the Browser

Akshat Gupta
Dept. of Computer Science & Engineering (AI)
KIET Group of Institutions
akshat.2226cseai1041@kiet.edu

Aishwarya Sinha
Dept. of Computer Science & Engineering (AI)
KIET Group of Institutions
aishwarya.2226cseai1@kiet.edu

Ms. Preeti Verma
Assistant Professor
Dept. of Computer Science & Engineering (AI)
KIET Group of Institutions
preeti.verma@kiet.edu

Dr. Shelly Gupta
Associate Professor
Dept. of Computer Science & Engineering (AI)
KIET Group of Institutions
shelly.gupta@kiet.edu

***Abstract*—For a long time students and researchers have used methods like 2D diagrams, textbook illustrations and mechanical orreries to understand celestial movement. These methods often prove inadequate when it comes to understanding 3D movements and overall trajectory and effects. These approaches usually lack engagement and learners often struggle when it comes to understanding three-dimensional relationships in celestial mechanics, such as elliptical orbits, axial tilts, and orbital periods. This paper presents a method where we use Threejs and WebGL to create a browser-based interactive orrery simulation that helps increase understanding of celestial mechanics through real-time 3D visualization. Using Three.js and WebGL,the system models planetary motion, orbital dynamics, and gravitational interactions which makes the movement of planets, their orbital paths, and related gravitational effects easier to follow. Users can speed up or slow down time, observe Kepler's laws in action, and explore different solar system phenomena at their own pace. The simulation combines dynamic data retrieval with interactive controls and small explanatory tooltips so that users can explore astronomical concepts more actively through a visual medium instead of only reading about them. The paper first explains the motivation behind building the system and what design decisions shaped it. It then reviews existing visualization tools and describes how the application was implemented using Three.js. The discussion also considers how interactive 3D environments can support learning and lastly,the system is evaluated and several possible improvements are suggested for future development. Also, an AI-based conversational assistant that relies on a Retrieval-Augmented Generation (RAG) technique is integrated to offer on-demand explanations of astronomical concepts, resulting in more interaction with and conceptual understanding between users.**

***Index Terms*—Celestial mechanics education, Browser-based simulation, Orbital dynamics, Keplerian laws, Educational technology, Real-time interaction.**

I. Introduction

Understanding how planets, moons, and other celestial bodies move has been an area of fascination for scientists and students for centuries. Celestial mechanics is a branch of astronomy that deals with the motions and gravitational interactions of these bodies, supports modern space exploration, satellite deployment, and our understanding of the architecture of the solar system. However, many students find the ideas like abstract nature of orbital dynamics and the mathematical complexity of gravitational interactions hard to imagine. Recent work in web technologies have made it easier to turn the browser into a powerful platform for interactive 3D visualization. With libraries such as Three.js and WebGL it now is possible to draw complex 3D scenes in real time, something that previously required specialized programs or strong graphics hardware. These tools have already been successfully applied in various domains, from immersive virtual gardens to interactive data visualizations, demonstrating their potential to make abstract scientific concepts more tangible and engaging. Even with this,there is still a lack of accessible learning tools for understanding celestial mechanics. Traditional orreries mechanical models of the solar system have for a long time been used as a valuable teaching aids, but they do not easily convey the differences in orbital speeds, changes in perspective, or how gravitational influence actually shapes a path. A browser-based interactive orrery simulation can help reduce this gap by allowing users to visualize planetary motions in real time, control simulation parameters, and see how gravitational forces influence orbital behaviour. The project, An Interactive Orrery Simulation for Learning Celestial Mechanics in the Browser, uses Three.js and WebGL to build a dynamic and accessible educational tool that runs directly in the browser. Concepts that are normally described in words or formulas—Kepler's laws, resonances between orbits, can be difficult to visualize. This becomes easier to understand when the user can manipulate the system directly. The interactive interface allows users to adjust time scales, switch perspectives, and access detailed information about each celestial object, thus turning passive observation into active exploration. The primary goal of this simulation is to make celestial mechanics accessible and engaging for a broad audience, including students, educators, and astronomy enthusiasts.By providing an engaging and interactive learn-

ing experience, the project seeks to bridge the gap between theoretical knowledge and intuitive understanding, fostering curiosity and deeper insight into the mechanics that govern our solar system. To make the system even more accessible and interactive, it uses a conversational interface whereby users may ask astronomical principles in any natural language. This will help learners to balance the visual exploration using the textual explanations, making the learning process more holistic.

II. Literature Review

It is clear from examining a large number of research studies and interactive astronomy platforms that a number of methods have been developed for the purpose of teaching and visualising celestial mechanics in an approachable way.Most available systems tend to fall into one of two categories: either they prioritise broad public engagement through interactive visuals, or they focus on high-accuracy scientific modelling, each with unique advantages and disadvantages.

In recent years, NASA's *Eyes on the Solar System* [9], [10] has gained popularity as an interactive tool. With the use of precise planetary ephemerides and actual mission data.It allows users to explore the solar system in 3D, follow spacecraft trajectories, examine alignments, and shift through time with ease. But users are only able to navigate pre-established scenarios and observe celestial events; they are unable to alter gravitational parameters or create their own orbital systems, which restricts their ability to understand celestial mechanics practically.

Another well-known set of tools is provided by the *PhET's My Solar System* [12], [13],that allow users to adjust bodies' masses, velocities, and initial positions, which makes them effective for introducing Newtonian gravity. These materials, which clearly depict fundamental gravitational interactions, are frequently used in physics education. But they frequently use oversimplified physics engines and lack the kind of accuracy and spatial depth needed to study more detailed orbital behaviour.

With precise ephemerides and interactive observation tools, detailed desktop programs such as *SpaceEngine* and *Starry Night* [1] offers photorealistic environments with highly accurate orbital models. These are commercial programs that require installation, powerful hardware, and sometimes paid licenses, which restricts their use in public or classroom settings despite their scientific accuracy.

Browser-based 3D frameworks like *Three.js* and *WebGL* have been investigated recently for scientific and educational visualisation. WebGL physics frameworks were evaluated by Munirov [8] and Yogya and Kosala [15], who confirmed that even on modest hardware, lightweight GPU-accelerated rendering can achieve smooth real-time simulation. Similar to this, Kitching et al. [7] and Fluke et al. [4] showed how interactive 3D graphics can be used to visualise astronomical datasets directly in the browser, reducing the gap between open education and professional visualisation.

This trend is reflected in newer open-source projects as well. In order to show how much the scalability of browser-side 3D rendering for large datasets is, Gurvich and Geller [6] created *Firefly*, an interactive visualisation based on WebGL that can render millions of particles in the browser. Salim et al. [14] and Pereira [11] demonstrated how browser-based simulations created with *Three.js* can successfully teach physics concepts through interactive models that proved effective in learning abstract concepts. Similarly, Ervana et al. [3] and Gattupalli [5] emphasised the educational potential of 3D simulations in enhancing student understanding and performance in STEM related fields.

Even with these developments, the majority of systems still do not integrate immersive 3D environments, real-time physics, and structured educational scaffolding. Usually, they are made for simplified experimentation or passive exploration. There is still much to learn about a browser-based orrery that combines the benefits of these approaches:

- Immersion exploration is made possible by realistic 3D visualisation and interactive controls.
- Hands-on experiments are supported by real-time gravitational simulation.
- Dynamic overlays and contextual explanations improve comprehension of celestial mechanics.

By utilising *Three.js* and *WebGL* for rendering and a client-side physics engine for real-time gravitational computation, the proposed *Interactive Orrery Simulation for Learning Celestial Mechanics in the Browser* seeks to close this gap. The system bridges the gap between professional astronomy visualisation and easily accessible, inquiry-based learning by integrating dynamic datasets, educational tooltips, and modular architecture.

III. Methodology

A. Technical Implementation

1) 3D Rendering and Visualization: To ensure cross-browser compatibility and take advantage of hardware acceleration for maximum performance, the simulation renders celestial bodies and orbital paths in real time using Three.js and WebGL.

- Celestial Models: Planets and moons are represented as sphere geometries within the 3D scene. To maintain visual clarity, planetary radii are scaled to approximately 1:1,000,000. Planetary Data System of NASA offers semi-solid surface features,like the Blue of the Earth. Normal maps and glittery highlights are used in dynamic lighting conditions to enhance further surface detail.
- Orbital Trails: The Orbital paths are represented with the help of`BufferGeometry` line objects, which are selected based on the reason that they manage regular updates effectively and with minimum memory overhead. Each orbit is colour-coded (such as red for Mars, yellow for Mercury, etc.) to differentiate planets easily incase the paths overlap. To avoid clutter on the screen, a

TABLE I: Comparison of Existing Astronomy Simulation Tools and the Proposed Interactive Orrery

System	Key Features	Limitations	Proposed Orrery Contribution
NASA Eyes on the Solar System	Correct planetary ephemerides, spacecraft trajectory visualization, interactive 3D exploration.	The user is expected to be able to have only preset scenarios but not be able to adjust system parameters or develop custom orbital systems.	A physical parameter editor (enabling experimental learning of the celestial mechanics) because it allows users to edit physical parameters (masses, velocities, positions, etc.).
PhET My Solar System	Educational simulation demonstrating Newtonian gravity with parameters that can be changed.	Simplified physics models are used and it does not have an immersive 3D graphics and realistic orbital behaviour.	Features three-dimensional (3D) real-time rendering and real-time physics simulation based on Three.js.
SpaceEngine / Starry Night	Very precise astronomic models and realistic photorealistic settings.	Commercial programs, which have to be downloaded and installed, high-level hardware, and even subscription fees.	Web-based platform with no software installation needed and can be used on an ordinary platform.
WebGL / Three.js Visualizations	Browser rendering with GPU to allow real-time graphics.	Most of the implementations are based on visualizations but do not include any physics or learning aids.	Combines graphic display and real-time simulation of gravity and educational overlays.
Firefly and similar WebGL projects	Interactive browser-based rendering of large astronomical datasets.	Its emphasis is on visualization as opposed to conceptual learning.	Adds contextual explanations and guided exploration for better understanding.
Proposed Interactive Orrery	Live simulation of gravity, 3D visualization simulator and Web access.	–	Snaps that divide professional astronomical visualization and interactive education tools.

proximity-based dynamic fading mechanism in a camera is employed that maintains spatial context.

- Solar light is mimicked using a directional light sourced at the position of the Sun. Light intensity is scaled according to the inverse-square relationship between luminosity and distance. Ambient lighting is added to ensure visibility of the scene across all areas. To further simulate the background environment, a deep-space environment is made up of a procedurally generated starfield that exists in roughly 10,000 particle points.

2) Orbital Mechanics: The movement of planets is calculated with the help of Newtonian dynamism of gravity. The gravitational force between bodies is defined as

$$\mathbf{F}_{ij} \;=\; G\,\frac{m_i m_j}{\|\mathbf{r}_{ij}\|^3}\,\mathbf{r}_{ij}, \tag{1}$$

where $\mathbf{r}_{ij}$ is the displacement of body i to body j [2].

The simulation relies on the Verlet integration method in order to update positions across time. Verlet integration is stable and offers better conservation of energy than the Euler method. The position update step is provided by

$$\mathbf{x}(t+\Delta t) \;=\; 2\mathbf{x}(t) - \mathbf{x}(t-\Delta t) + \mathbf{a}(t)\,\Delta t^2. \tag{2}$$

To minimize the threat of instability in extreme time acceleration, the timestep Δt is put at 60m.

3) Time Control: The user interface has a logarithmic slider that enables the alteration of the simulation speed between $1\times$ to $10^6\times$ in real time. A time-warp factor α dynamically scales the simulation timestep:

$$\Delta t_{\text{sim}} = \alpha \cdot \Delta t_{\text{real}}.$$

This mechanism allows users to view long-term orbital behaviour—as of planetary alignments or orbital precession—while also keeping the application responsive and interactive at the same time.

B. System Architecture

1) Client-Side Workflow: The simulation follows a four-layer architecture that is executed completely inside the browser, removing the need for server-side computation.

1) **Data Layer:** Planetary and orbital parameters are stored in JSON files and loaded in the initiation of the application. Some of the properties present in these files include mass, radius and orbital elements. These data undergo prior processing with the information from NASA's JPL Horizons system. For time intervals that range past the year 3000 CE, orbital values are extended using secular perturbation models so that the system remains continuous even over very long time spans.
2) **Physics Layer:** Gravitational interactions are computed in a dedicated Web Worker thread to prevent blocking the main UI thread. In the case when the simulation has more calculations of the force than that of 100 celestial bodies,the processing is done in batches to maintain steady performance.
3) **Rendering Layer:** The 3D scene is controlled through the scene graph presented by Three.js. Several optimizations are used during rendering. Performance is improved through frustum culling (to forgo rendering off-screen objects) and occlusion queries, which stops rendering of objects hidden behind others.
4) **UI Layer:** React is used to implement the interface simulation controls and application, which has made the application state to remain modular and reactive. Interactive elements, such as sliders and touch gestures,

has made it possible to simulate the parameters of control intuitively.

5) **AI Interaction Layer:** This is another layer that is added to facilitate verbal communication with a chatbot interface. This layer interprets queries posed by the users, retrieves the information related to the query in a local knowledge base, and produces context-sensitive responses. It serves as an interface between the user interface and the data layer which allows users to interact with astronomical concepts via natural language dialogue.

2) Performance Optimization:

- **Level of Detail (LOD):** The rendering is made less complex by making things far away simpler. Asteroids located beyond the distance of about 10 astronomical units (AU) are displayed as 2D billboard sprites rather than full 3D meshes. Additionally, shadow mapping is not enabled for bodies with diameters below 1,000 km. This is because they have very less visual impact at that scale.
- **Memory Management:** Orbital trails are restricted to approximately ten revolutions for each body. As the simulation runs , the older segments are gradually. This keeps the scene clear in sight while also saving memory that might have been consumed by simulations running for longer durations.

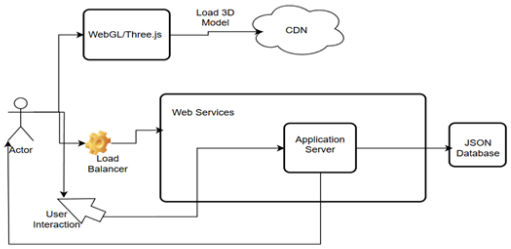

Fig. 1: Proposed System Architecture

C. User Interface Design

1) Control Panel: The simulation contains a control panel that allows users to interact with the system and modify various parameters as we discover the solar system. The layout was made in a way that enables fast observation and also more detailed possible.

- **Time Controls:** The pace of the simulation can be adjusted using a logarithmic slider. It ranges from real ($1\times$) to very high accelerations rate up to ($10^6\times$).Navigational ability in time is made possible through snap points at significant moments(such as one day/second and one year/second). There is also a inbuilt date selector that works in both Julian and Gregorian calendar formats. This facilitates analysis of events that happened either a long time ago or will happen in the future.
- **Camera Modes:** There are three camera modes that are available in the simulation that provide navigation in the simulation to allow the users to navigate through the system.
  - **Heliocentric Mode:** This mode involves movement of the camera around the Sun in circles at a constant speed of approximately 15 degrees per second. The view gives a broad overview of the solar system and makes planetary alignments and their movement easier to observe.
  - **Chase Mode:** It is also possible to lock the camera to an object chosen, such as a planet, moon, or spacecraft. The camera then follows the object in a fixed manner at a distance of roughly five kilometres. This view is especially helpful for closely tracking individual trajectories or seeing how the objects interact with other bodies.
  - **Free Mode:** Free mode the camera is controlled manually by the user. Movement uses standard WASD keyboard navigation and mouse orientation inputs.The users is free to move anywhere in the scene and examine at any simulation viewpoint they choose.

D. Database Design

In this project, astronomical data is contained in a JSON file called `celestialBodies.json`,rather than being handled by using a conventional database system. Since the application fully run in the browser, we simply required a minimalist manner to hold sticky data of the objects in the simulation.The planets, moons, and other entries of the bodies are in the JSON file along with properties such as name, object type, mass, radius, and albedo. Orbital parameters—like semi-major axis, eccentricity, inclination, and epoch—are also in the JSON file so the physics module easily fetch the data and calculate their motion. Using a JSON file makes loading and processing decisions within the browser application easy. When the simulation starts, the file is read and the values are passed to the rendering and physics components implemented with Three.js and WebGL. Because everything runs on the client side, delays that would come from queries or network requests to database are avoided and the simulation runs faster.

The second reason of practicality of using JSON is that it makes the data easy to modify. In the course of development we had to change orbital parameters or insert test objects. With a simple JSON structure, those changes can be made directly in the file without altering the rest of the code thus reducing the chance of accidently introducing issues and bugs in the code. This also makes it possible to extend the simulation later if additional bodies or new astronomical values are provided.

A whole database system was not deemed necessary for the scope of the project. The simulation does not require complex queries, user accounts, or large datasets, so adding a database would add complexity and maintenance to the system.. Keeping the data in a JSON file allows the application

to be uncomplicated yet give all the information required to simulate the orrery.

E. Experimental Setup

All experiments and performance evaluations for our project Interactive Orrey Simulation were carried out on a desktop computer with 16GB of RAM, which provided sufficient memory for running the browser-based simulation alongside development tools. The desktop is also equipped with a 1TB hard disk drive (HDD) for storing large files and generating simulation outputs.

The computer is powered by an Intel 10th Generation Core i5 processor, which that performs the majority of the calculations. Tasks such as real-time rendering, physics calculations, and browser execution are processed on the CPU. The computer has integrated Intel UHD Graphics or Intel Iris Xe Graphics for graphics processing, and an NVIDIA GeForce MX350 GPU can be used to add more support while rendering simulations. All these hardware configuration are enough to run the simulation smoothly without any hiccups, even in hard and demanding scenarios.

The system is suitable for both desktop and portable use due to its small size, measuring 359.2 $\times$ 235.8 $\times$ 19.9 mm and weighing 1.7 kg at startup. This hardware setup also aligns well with the requirements for Three.js and WebGL-based applications, as recommended in the community, and thus proves to be a strong platform in both user-testing and development of the browser based orrery simulation.

In Figure 2 we can see the whole process flow of an Interactive Orrery Simulation.The first stage, which is the initialization stage, the required libraries are loaded and the 3D rendering engine is configured. The system then retrieves celestial body data from local data files it was provided and constructs the simulation environment.The UI setup part starts with first creating the background along with grid, and then 3D models of planets and other celestial bodies are loaded and rendered into the system using the JSON data. After the setup, users will have the ability to model objects to see orbital dynamics, zoom and spin around in a better view of the solar system, or pick certain planets to learn more about them.The simulation continuously updates the scene in response to user input.The simulation continuously retrieves the scene based on the input by the user. When one of the celestial objects is chosen, and the system reacts to it by either bringing more information on the chosen objects or the 3D view needs to be updated by the user interactions,developing a virtual planetary exploration.

F. AI-based Conversational Assistant (RAG Chatbot)

In order to make the interactive orrery simulation educationally more efficient, an AI-powered conversational assistant is introduced into the system. The chatbot allows the users to communicate with the simulation through natural language questions and receive answers to astronomical questions in real-time.

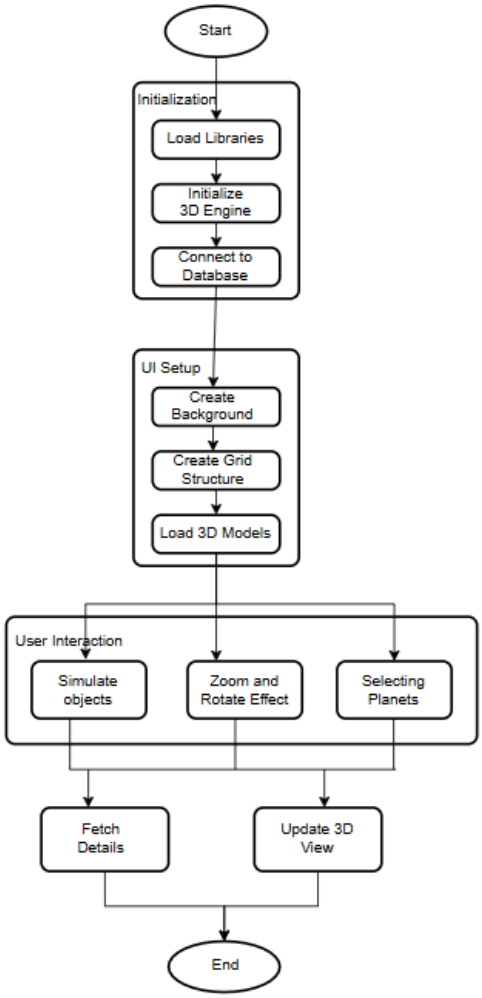

Fig. 2: Flowchart for Proposed System

The system uses a Retrieval-Augmented Generation (RAG) developed where responses are constructed using information recalled out of a structured body of knowledge instead of using pre-learned responses or predefined responses. This makes responses to be domain-specific and precise.

1) **Construction of Knowledge Base Curation:** The knowledge base will be constructed based on educational content that was curated based on standard academic sources, such as Curtis, "Orbital Mechanics for Engineering Students." [2]. The most important ideas like the laws of Kepler, the forces of gravity, the motion of the orbs as well as the characteristics of the planets are torn off and stored as text fragments.
2) **Text Chunking and Storage:** The extracted information is broken down into smaller parts (chunks) to enable retrieval. Such pieces can be stored on the local browser such that they can be accessed quickly without processing on the server.
3) **Information Retrieval:** Giving a query, the system

searches through the stored text chunks by making a similarity search. Relevant segments are chosen on the basis of matching of key words and overlap of context to the query of the user.

4) **Response Generation:** The segments retrieved are re-assembled into an effective response. This will make sure that the chatbot gives an explanation based on scientific facts, as opposed to creating unsubstantiated information.

The chatbot is embedded as part of the UI layer and it is all client-side, which retains the lightweight and browser-based character of the application. Interactive visualization combined with conversational querying helps the system to encourage more engagement and comprehension of celestial mechanics.

IV. Results and Discussion

Interactive Orrery Simulation has a variety of major and minor celestial bodies, each with a little information panel containing important details on the object. These panels describe fundamental physical properties, orbital, and some scientific facts that will make the users aware of their role in the solar system. Figure 3,as you see,illustrates Earth, along with details such as its axial tilt and orbital period.Figure 4,has an overview of the solar system from the top which allows users to see the positions of all the planets and how they move along their orbits.In Figure 5 we can see snapshot of a satellite traveling in the simulation and eventually passing near Jupiter,where the trajectory slightly changes demonstrating how movement of bodies can change with the strong gravitational influence of large planets. Users can open the information panels for each object or scenario to read additional explanations as to what is happening and why. They can also follow orbital motion as it happens in the simulation, and perceive related astronomical and physics concepts through the information provided as the simulation runs in the background.

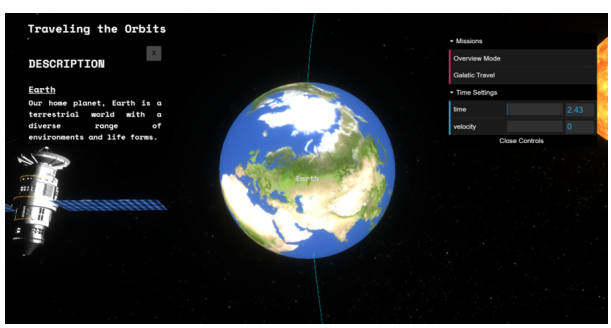

Fig. 3: Earth 3D Model

A. Performance Metrics

The simulation performed outstandingly across different hardware setups during testing. When tested on a mid-range desktop (Intel i5, 16 GB RAM, NVIDIA MX350 GPU), the system kept in the vicinity of 55–60 FPS in rendering the inner solar system with planets and their satellites. During heavier scenarios where there are multiple satellites moving at once, the performance dropped slightly but still stayed between 38 and 45 FPS,which is understandable.Even during this, the simulation responsive, which is good. On an SSD, initial scene load times averaged 1.8 seconds, whereas more complicated scenarios took 2.5 seconds. Web performance metrics also reflected this responsiveness, with LCP at 0.12 s, CLS at 0, and INP at 32 ms, indicating that the simulation was able to render scenes fast and was able to give out stable simulations to the user which is needed. These outcomes can easily show how well the client-side architecture and WebGL optimizations, such as frustum culling and Level of Detail (LOD) rendering work and how the Orrey simulation benefits from it by giving fast and stable simulations.

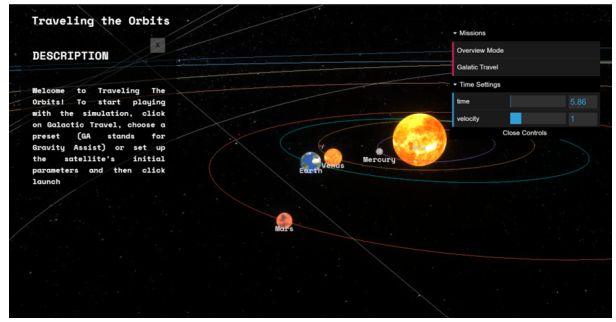

Fig. 4: Comprehensive overview of the solar system

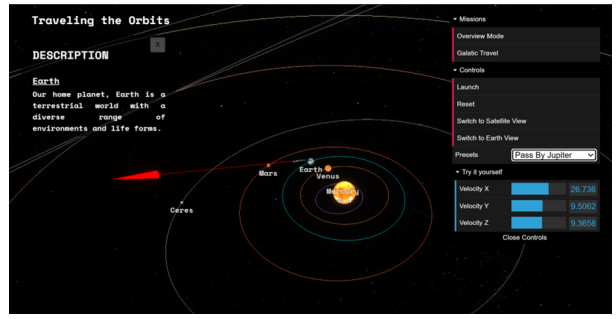

Fig. 5: Simulated movement of satellite to pass by Jupiter

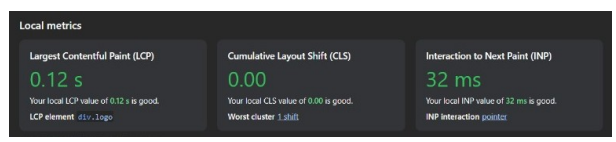

Fig. 6: Local Metrics for the 3D Solar System

B. Comparison with Existing Tools

The n-body physics engine of this simulation is real time which enables itsusers to make and experiment their scenarios. It is not the same as NASA's Eyes on the Solar System, where trajectories are precalculated in the majority of cases. For example, in this simulation, a user can change parameters such as Earth's mass and immediately observe how the surrounding orbits change. Additionally,the JSON-based data architecture allows great flexibility than other databases as movements

can be calculated using simple data quickly rather than using queries in an actual database. New celestial bodies can be introduced without major changes to the program, which is more flexible than relying on fixed or static datasets. However,the simulation is yet to be scientifically accurate. Because it is based upon simplified Newtonian physics, the results are not designed to be used in astronomical research of high precision.

C. Chatbot Evaluation

The reliability of the RAG-based conversational assistant was considered qualitatively on the capacity to retrieve and present pertinent information in the knowledge base. A sequence of representative questions concerning planar mechanics and planetary science was challenged. The chatbot was shown to be consistent in responding to questions of domain specificity because it accesses the proper contextual data in the knowledge base based on the available academic literature. The system, however, can only be as effective as the available data that is accessible to it and could not respond to out-of-domain queries.

TABLE II: Sample Chatbot Evaluation Results

Query	Expected Output	System Response
What is Mars?	Planet description	Correct planetary info
Explain Kepler's First Law	Orbital law explanation	Correct explanation
What is orbital velocity?	Definition	Relevant explanation
What is artificial intelligence?	Out of scope	Rejected / fallback response

The chatbot gave the right and contextually appropriate responses to in-domain queries and rightly declined the out-of-domain queries.

V. Conclusion

The Interactive Orrery Simulation was created to observe the way a web-based tool may be used to illustrate some of the concepts from celestial mechanics. Three.js is utilized in the system to render the planets and orbits of the planets in 3D, as well as a plain client-side physics causes their motion to be updated in real time. The majority of the planetary parameters and properties of objects were easier to develop because it was stored in JSON files making it easy to make changes or introduce new bodies without altering much of the code.

This project presents a visual way for learners to understand abstract ideas and theories that are not always easily pictured. Future improvements can also be incorporated in the project, like adding VR support helping in creating a planetarium like experience or creating simulations for future missions based on scientific data, despite the limitations due to simplified physics model and performance restrictions on low-end hardware that it currently have.The project shows how modern web technologies can be used to build interactive tools that helps people explore astronomy concepts in a more hands-on way and provide ways for an accessible transformative education. The introduction of a Retrieval-Augmented Generation (RAG)-based conversational assistant can only serve as a clear example of how intelligent-based information retrieval may be utilized along with interactive visualization for better learning results.

References

[1] Celestron. Celestron starry night software, 2024. Available at: https://www.celestron.com/pages/celestron-starry-night-software.

[2] H. D. Curtis. *Orbital Mechanics for Engineering Students*. Elsevier, 3rd edition, 2016.

[3] D. S. Ervana, R. Raharjo, and M. Munasir. The effectiveness of 3d models online modules to practice mastery of solar system conceptual knowledge. *International Journal of Recent Educational Research (IJORER)*, 3(2):162–181, 2022.

[4] C. J. Fluke, D. G. Barnes, and N. T. Jones. Interchanging interactive 3-d graphics for astronomy. *arXiv preprint arXiv:0810.4201*, 2008.

[5] S. Gattupalli. Browser-based educational simulations for physics learners, teachers, and science enthusiasts everywhere, 2025. Available at: https://www.gattupalli.com/research/physicsed-simulations.

[6] A. B. Gurvich and A. Geller. Firefly: A browser-based interactive 3d data visualization tool for millions of data points. *arXiv preprint arXiv:2207.13706*, 2022.

[7] T. Kitching, A. Amara, A. Rassat, and A. Refregier. Cloud cosmology: Building the web interface for icosmo. *arXiv preprint arXiv:0901.3143*, 2009.

[8] J. J. Munirov. Three.js: A comprehensive overview of web-based 3d graphics framework. *Journal of Applied Science and Social Science*, 15(9):363–365, 2025.

[9] NASA Jet Propulsion Laboratory. Explore the solar system with nasa's new-and-improved 3d 'eyes', 2022. Available at: https://www.nasa.gov/centers-and-facilities/jpl/explore-the-solar-system-with-nasas-new-and-improved-3d-eyes/.

[10] NASA Jet Propulsion Laboratory. Eyes on the solar system, 2025. Available at: https://eyes.nasa.gov/apps/solar-system.

[11] K. J. Pereira. Water simulation on webgl and three.js. Master's thesis, University of Southern Mississippi, 2013. Available at: https://aquila.usm.edu/honors_theses/137/.

[12] PhET Interactive Simulations. My solar system, 2024. University of Colorado Boulder. Available at: https://phet.colorado.edu/en/simulation/my-solar-system.

[13] E. C. Prima et al. Learning solar system using phet simulation to improve students' understanding and motivation. *Journal of Science Learning*, 1(2), 2018.

[14] W. Salim, A. S. Setiawan, and Y. Purnama. Three-dimensional web-based physics simulation application for physics learning tool. *CommIT (Communication and Information Technology) Journal*, 6(2), 2012.

[15] R. Yogya and R. Kosala. Evaluation of physics frameworks for building web-based games. *Journal of Games, Game Art, and Gamification*, 2024. Bina Nusantara University, Jakarta, Indonesia.